COMPLETE CURSOR AI DEVELOPMENT PROMPT

One-to-One Tutoring Platform with Whiteboard & Chat

Version: 2.0 FINAL
Timeline: 12-16 Weeks
Last Updated: January 30, 2026

🎯 PROJECT OVERVIEW

What You're Building

A **web-based one-to-one tutoring platform** where:

- **Students** and **Teachers** register and get approved by admin
- **Admin** creates and assigns tutoring sessions
- **Participants** join video calls with **integrated whiteboard** and **chat**
- **Real-time collaboration** during sessions
- **No payment processing** (admin handles billing separately)

⚠️ CRITICAL CONSTRAINTS

YOU MUST FOLLOW THESE RULES

✅ ONLY Build What's Listed Below

❌ DO NOT Add These Features:

- ❌ Payment integration (Razorpay, Stripe, etc.)
- ❌ Automated booking by students
- ❌ Mobile apps (iOS/Android)
- ❌ Session recording
- ❌ SMS notifications
- ❌ Social media login
- ❌ Quiz/assessment tools
- ❌ Homework submission
- ❌ Parent portal
- ❌ Certificate generation
- ❌ AI features

- ❌ AI features
- ❌ Video filters/effects
- ❌ Virtual backgrounds

🔪 Exact Technology Stack (NO Substitutions)

Frontend:

```json
{
  "framework": "Next.js 14.2.0+",
  "language": "TypeScript 5.3.0+",
  "styling": "Tailwind CSS 3.4.0+",
  "components": "Shadcn/UI",
  "video": "@livekit/components-react@2.0.0+",
  "whiteboard": "@excalidraw/excalidraw@0.17.0+",
  "stateManagement": {
    "server": "@tanstack/react-query@5.0.0+",
    "client": "zustand@4.5.0+"
  },
  "forms": "react-hook-form@7.50.0+",
  "validation": "zod@3.22.0+",
  "dateTime": "date-fns@3.0.0+",
  "icons": "lucide-react@0.300.0+"
}
```

Backend:

```json
{
  "language": "Go 1.22+",
  "framework": "gin-gonic/gin@1.10.0+",
  "database": {
    "driver": "jackc/pgx/v5",
    "orm": "gorm.io/gorm@1.25.0+"
  },
  "cache": "redis/go-redis/v9",
  "auth": "golang-jwt/jwt/v5",
  "validation": "go-playground/validator/v10",
  "config": "spf13/viper",
  "livekit": "livekit/server-sdk-go",
  "migrations": "golang-migrate/migrate/v4",
  "email": "AWS SES SDK for Go"
}
```

Infrastructure:

```json
{
  "database": "PostgreSQL 15.0+",
  "cache": "Redis 7.2+",
  "video": "LiveKit Server latest",
  "storage": "AWS S3",
  "cdn": "AWS CloudFront",
  "email": "AWS SES",
  "containerization": "Docker + Docker Compose",
  "deployment": "AWS (ap-south-1 Mumbai region)"
}
```

📊 COMPLETE DATABASE SCHEMA

Tables (Exact Schema - DO NOT MODIFY)

1. Users Table

```sql
```

```sql
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email VARCHAR(255) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  first_name VARCHAR(100) NOT NULL,
  last_name VARCHAR(100) NOT NULL,
  role VARCHAR(20) NOT NULL CHECK (role IN ('student', 'teacher', 'admin')),
  phone VARCHAR(15),
  avatar_url TEXT,
  timezone VARCHAR(50) DEFAULT 'Asia/Kolkata',

  -- Timestamps
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  last_login TIMESTAMP WITH TIME ZONE,

  -- Status
  is_verified BOOLEAN DEFAULT FALSE,
  is_active BOOLEAN DEFAULT TRUE
);

CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_users_role ON users(role);
CREATE INDEX idx_users_active ON users(is_active);
```

2. Teachers Table

```sql
CREATE TABLE teachers (
  id UUID PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE,
  bio TEXT,
  subjects TEXT[] NOT NULL,
  experience_years INTEGER DEFAULT 0,
  is_approved BOOLEAN DEFAULT FALSE,

  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE INDEX idx_teachers_approved ON teachers(is_approved);
CREATE INDEX idx_teachers_subjects USING GIN (subjects);
```

3. Sessions Table

```sql
```

```sql
CREATE TABLE sessions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

  -- Room identifiers
  room_id VARCHAR(100) UNIQUE NOT NULL,
  livekit_room_name VARCHAR(200) UNIQUE NOT NULL,

  -- Participants
  student_id UUID NOT NULL REFERENCES users(id),
  teacher_id UUID NOT NULL REFERENCES tutors(id),

  -- Scheduling
  scheduled_start TIMESTAMP WITH TIME ZONE NOT NULL,
  scheduled_end TIMESTAMP WITH TIME ZONE NOT NULL,
  duration_minutes INTEGER NOT NULL,

  -- Status
  status VARCHAR(20) NOT NULL DEFAULT 'scheduled'
    CHECK (status IN ('scheduled', 'in_progress', 'completed', 'cancelled')),

  -- Session details
  subject VARCHAR(100) NOT NULL,
  notes TEXT,


  -- Timestamps
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE INDEX idx_sessions_student ON sessions(student_id, scheduled_start);
CREATE INDEX idx_sessions_teacher ON sessions(teacher_id, scheduled_start);
CREATE INDEX idx_sessions_status ON sessions(status);
CREATE INDEX idx_sessions_scheduled_start ON sessions(scheduled_start);
CREATE INDEX idx_sessions_room ON sessions(room_id);

-- Prevent double-booking
CREATE EXTENSION IF NOT EXISTS btree_gist;

ALTER TABLE sessions ADD CONSTRAINT no_teacher_overlap
  EXCLUDE USING GIST (
    teacher_id WITH =,
    tstzrange(scheduled_start, scheduled_end) WITH &&
  ) WHERE (status IN ('scheduled', 'in_progress'));

ALTER TABLE sessions ADD CONSTRAINT no_student_overlap
  EXCLUDE USING GIST (
    student_id WITH =,
```

```
  tstzrange(scheduled_start, scheduled_end) WITH &&
) WHERE (status IN ('scheduled', 'in_progress'));
```

4. Session Messages Table

sql

```sql
CREATE TABLE session_messages (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  session_id UUID NOT NULL REFERENCES sessions(id) ON DELETE CASCADE,
  sender_id UUID NOT NULL REFERENCES users(id),

  -- Message content
  message_type VARCHAR(20) NOT NULL CHECK (message_type IN ('text', 'file')),
  content TEXT NOT NULL,
  file_url TEXT,
  file_name VARCHAR(255),
  file_size INTEGER,

  -- Status (for read receipts)
  is_read BOOLEAN DEFAULT FALSE,
  read_at TIMESTAMP WITH TIME ZONE,

  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE INDEX idx_session_messages_session ON session_messages(session_id, created_at);
CREATE INDEX idx_session_messages_sender ON session_messages(sender_id);
CREATE INDEX idx_session_messages_unread ON session_messages(session_id, is_read);
```

```sql
CREATE INDEX idx_session_messages_unread ON session_messages(session_id, is_read);
```

5. Whiteboard States Table

```sql
CREATE TABLE whiteboard_states (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  session_id UUID NOT NULL REFERENCES sessions(id) ON DELETE CASCADE,

  -- Whiteboard data
  state_data JSONB NOT NULL,
  version INTEGER NOT NULL DEFAULT 1,

  -- Metadata
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  created_by UUID NOT NULL REFERENCES users(id)
);

CREATE INDEX idx_whiteboard_session ON whiteboard_states(session_id, version);
CREATE INDEX idx_whiteboard_created_at ON whiteboard_states(created_at);
```

6. Session Files Table

```sql
CREATE TABLE session_files (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  session_id UUID NOT NULL REFERENCES sessions(id) ON DELETE CASCADE,
  uploaded_by UUID NOT NULL REFERENCES users(id),

  -- File info
  file_name VARCHAR(255) NOT NULL,
  file_url TEXT NOT NULL,
  file_size INTEGER NOT NULL,
  file_type VARCHAR(50) NOT NULL,

  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE INDEX idx_session_files_session ON session_files(session_id, created_at);
CREATE INDEX idx_session_files_uploader ON session_files(uploaded_by);
```

7. Notifications Table

```sql
CREATE TABLE notifications (
```

```
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,

  -- Notification content
  type VARCHAR(50) NOT NULL,
  title VARCHAR(255) NOT NULL,
  message TEXT NOT NULL,

  -- Related entities
  related_session_id UUID REFERENCES sessions(id),

  -- Status
  is_read BOOLEAN DEFAULT FALSE,
  read_at TIMESTAMP WITH TIME ZONE,

  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE INDEX idx_notifications_user ON notifications(user_id, is_read, created_at);
CREATE INDEX idx_notifications_session ON notifications(related_session_id);
```

🔌 COMPLETE API SPECIFICATION

Base URL Structure

```
Development: http://localhost:8080/api/v1
Production: https://api.yourdomain.com/api/v1
```

Authentication Header

```http
Authorization: Bearer {jwt_token}
```

API Endpoints (ALL ENDPOINTS - DO NOT ADD MORE)

Authentication

1. Register User

```http
POST /api/v1/auth/register
```

```
Request Body:
{
  "email": "user@example.com",
  "password": "SecurePass123!",
  "firstName": "John",
  "lastName": "Doe",
  "role": "student",  // or "teacher"
  "phone": "+919876543210"
}

Response: 201 Created
{
  "user": {
    "id": "uuid",
    "email": "user@example.com",
    "firstName": "John",
    "lastName": "Doe",
    "role": "student",
    "isVerified": false,
    "isActive": true
  },
  "token": "eyJhbGc..."
}

Errors:
- 400: Validation error (email invalid, password too weak, etc.)
- 409: Email already exists
```

2. Login

```
http
POST /api/v1/auth/login
```

```
Request Body:
{
  "email": "user@example.com",
  "password": "SecurePass123!"
}

Response: 200 OK
{
  "user": { ... },
  "token": "eyJhbGc..."
}

Errors:
- 400: Invalid credentials
- 403: Account not active or not approved (for teachers)
```

3. Logout

```http
POST /api/v1/auth/logout
Authorization: Bearer {token}

Response: 200 OK
{
  "message": "Logged out successfully"
}
```

Users

4. Get Current User

```http
GET /api/v1/users/me
Authorization: Bearer {token}

Response: 200 OK
{
  "id": "uuid",
  "email": "user@example.com",
  "firstName": "John",
  "lastName": "Doe",
  "role": "student",
  "phone": "+919876543210",
  "avatarUrl": null,
  "timezone": "Asia/Kolkata",
  "isVerified": true,
```

```
  "isVerified": true,
  "isActive": true,
  "createdAt": "2026-01-30T10:00:00Z",
  "lastLogin": "2026-01-30T15:30:00Z"
}
```

5. Update Current User

```http
PATCH /api/v1/users/me
Authorization: Bearer {token}

Request Body:
{
  "firstName": "John",
  "lastName": "Smith",
  "phone": "+919876543211",
  "timezone": "Asia/Kolkata"
}

Response: 200 OK
{
  "id": "uuid",
  "email": "user@example.com",
  "firstName": "John",
  "lastName": "Smith",
  ...
}
```

Teachers

6. List Teachers

```http
```

```
GET /api/v1/teachers?subject=Math&page=1&limit=10
Authorization: Bearer {token}

Query Parameters:
- subject (optional): Filter by subject
- page (optional): Page number (default: 1)
- limit (optional): Items per page (default: 10)

Response: 200 OK
{
  "teachers": [
    {
      "id": "uuid",
      "firstName": "Jane",
      "lastName": "Teacher",
      "bio": "Experienced math tutor...",
      "subjects": ["Math", "Physics"],
      "experienceYears": 5,
      "avatarUrl": null
    }
  ],
  "total": 25,
  "page": 1,
  "limit": 10,
  "totalPages": 3
}
```

7. Get Teacher Details

http

```
GET /api/v1/teachers/:id
```

```
Authorization: Bearer {token}

Response: 200 OK
{
  "id": "uuid",
  "firstName": "Jane",
  "lastName": "Teacher",
  "bio": "Experienced math tutor with 5 years...",
  "subjects": ["Math", "Physics"],
  "experienceYears": 5,
  "avatarUrl": null,
  "isApproved": true
}
```

Sessions

8. List User Sessions

```http
GET /api/v1/sessions?status=scheduled&startDate=2026-02-01&endDate=2026-02-28
Authorization: Bearer {token}

Query Parameters:
- status (optional): Filter by status (scheduled/in_progress/completed/cancelled)
- startDate (optional): ISO 8601 date
- endDate (optional): ISO 8601 date
```

- page, limit (pagination)

Response: 200 OK

```
{
  "sessions": [
    {
      "id": "uuid",
      "roomId": "session-abc123",
      "subject": "Math",
      "scheduledStart": "2026-02-01T10:00:00Z",
      "scheduledEnd": "2026-02-01T11:00:00Z",
      "durationMinutes": 60,
      "status": "scheduled",
      "teacher": {
        "id": "uuid",
        "firstName": "Jane",
        "lastName": "Teacher"
      },
      "student": {
        "id": "uuid",
        "firstName": "John",
        "lastName": "Doe"
      }
    }
  ],
  "total": 10,
  "page": 1
}
```

9. Create Session (Admin Only)

```
http
```

POST /api/v1/sessions
Authorization: Bearer {admin_token}

Request Body:
```
{
  "studentId": "uuid",
  "teacherId": "uuid",
  "subject": "Math",
  "scheduledStart": "2026-02-01T10:00:00Z",
  "durationMinutes": 60,
  "notes": "Focus on algebra"
}
```

Response: 201 Created
```
{
  "id": "uuid",
  "roomId": "session-abc123",
  "livekitRoomName": "session-abc123",
  "studentId": "uuid",
  "teacherId": "uuid",
  "subject": "Math",
  "scheduledStart": "2026-02-01T10:00:00Z",
  "scheduledEnd": "2026-02-01T11:00:00Z",
  "durationMinutes": 60,
  "status": "scheduled",
  "notes": "Focus on algebra",
  "createdAt": "2026-01-30T10:00:00Z"
}
```

Errors:
- 400: Validation error
- 403: Not authorized (must be admin)
- 409: Teacher or student has conflicting session

10. Get Session Details

```http
GET /api/v1/sessions/:id
Authorization: Bearer {token}

Response: 200 OK
{
  "id": "uuid",
  "roomId": "session-abc123",
  "subject": "Math",
  "scheduledStart": "2026-02-01T10:00:00Z",
  "scheduledEnd": "2026-02-01T11:00:00Z",
  "durationMinutes": 60,
  "status": "scheduled",
  "notes": "Focus on algebra",
  "teacher": {
    "id": "uuid",
    "firstName": "Jane",
    "lastName": "Teacher",
    "avatarUrl": null
  },
  "student": {
    "id": "uuid",
    "firstName": "John",
    "lastName": "Doe",
    "avatarUrl": null
  }
}

Errors:
- 403: Not authorized (must be participant or admin)
- 404: Session not found
```

11. Update Session (Admin Only)

```http
PATCH /api/v1/sessions/:id
Authorization: Bearer {admin_token}

Request Body:
{
```

```
  "scheduledStart": "2026-02-01T11:00:00Z",
  "durationMinutes": 90,
  "notes": "Extended session for exam prep"
}

Response: 200 OK
{
  "id": "uuid",
  ...updated fields...
}

Errors:
- 403: Not authorized
- 404: Session not found
- 409: New time conflicts with other sessions
```

12. Cancel Session (Admin Only)

```http
DELETE /api/v1/sessions/:id
Authorization: Bearer {admin_token}

Response: 200 OK
{
  "message": "Session cancelled successfully"
}
```

13. Join Session

```http
POST /api/v1/sessions/:id/join
Authorization: Bearer {token}
```

```
Authorization: Bearer {token}

Response: 200 OK
{
  "livekitToken": "eyJhbGc...",
  "livekitUrl": "wss://livekit.yourdomain.com",
  "roomName": "session-abc123",
  "session": {
    "id": "uuid",
    "subject": "Math",
    "scheduledStart": "2026-02-01T10:00:00Z",
    "scheduledEnd": "2026-02-01T11:00:00Z",
    "otherParticipant": {
      "id": "uuid",
      "name": "Jane Teacher",
      "role": "teacher",
      "avatarUrl": null
    }
  }
}

Errors:
- 400: Cannot join yet (too early) or session ended
- 403: Not authorized (must be participant)
- 404: Session not found
```

14. Update Session Status

```http
PATCH /api/v1/sessions/:id/status
Authorization: Bearer {token}

Request Body:
{
  "status": "in_progress"  // or "completed"
}

Response: 200 OK
{
  "id": "uuid",
  "status": "in_progress"
}
```

Messages

15. Get Session Messages

http

```
GET /api/v1/sessions/:sessionId/messages?page=1&limit=50
Authorization: Bearer {token}

Response: 200 OK
{
  "messages": [
    {
      "id": "uuid",
      "sessionId": "uuid",
      "senderId": "uuid",
      "senderName": "John Doe",
      "messageType": "text",
      "content": "Hello, teacher!",
      "isRead": true,
      "readAt": "2026-02-01T10:15:00Z",
      "createdAt": "2026-02-01T10:05:00Z"
    },
    {
      "id": "uuid",
      "messageType": "file",
      "content": "Shared a file",
      "fileUrl": "https://s3.amazonaws.com/...",
      "fileName": "homework.pdf",
      "fileSize": 102400,
      "createdAt": "2026-02-01T10:10:00Z"
    }
  ],
  "total": 25,
  "page": 1
}
```

16. Send Message

http

POST /api/v1/sessions/:sessionId/messages
Authorization: Bearer {token}

Request Body (Text):
```
{
  "messageType": "text",
  "content": "Hello, teacher!"
}
```

Request Body (File):
```
{
  "messageType": "file",
  "content": "Shared a file",
  "fileUrl": "https://s3.amazonaws.com/...",
  "fileName": "homework.pdf",
  "fileSize": 102400
}
```

Response: 201 Created
```
{
  "id": "uuid",
  "sessionId": "uuid",
  "senderId": "uuid",
  "messageType": "text",
```

```
  "content": "Hello, teacher!",
  "createdAt": "2026-02-01T10:05:00Z"
}
```

17. Mark Messages as Read

```http
PATCH /api/v1/sessions/:sessionId/messages/read
Authorization: Bearer {token}

Request Body:
{
  "messageIds": ["uuid1", "uuid2", "uuid3"]
}

Response: 200 OK
{
  "updated": 3
}
```

Whiteboard

18. Get Whiteboard State

```http
GET /api/v1/sessions/:sessionId/whiteboard/latest
Authorization: Bearer {token}

Response: 200 OK
{
  "id": "uuid",
  "sessionId": "uuid",
  "stateData": {
    "elements": [...excalidraw elements...],
    "appState": {...excalidraw app state...}
  },
  "version": 5,
  "createdAt": "2026-02-01T10:30:00Z",
  "createdBy": "uuid"
}

Errors:
- 404: No whiteboard state found
```

19. Save Whiteboard State

```http
POST /api/v1/sessions/:sessionId/whiteboard
Authorization: Bearer {token}

Request Body:
{
  "stateData": {
    "elements": [...excalidraw elements...],
    "appState": {...excalidraw app state...}
  }
}

Response: 201 Created
{
  "id": "uuid",
  "sessionId": "uuid",
  "version": 6,
  "createdAt": "2026-02-01T10:35:00Z"
}
```

20. Export Whiteboard

```http
POST /api/v1/sessions/:sessionId/whiteboard/export
Authorization: Bearer {token}

Request Body:
{
  "format": "png"  // or "svg"
}

Response: 200 OK
{
  "exportUrl": "https://s3.amazonaws.com/.../whiteboard-export.png",
  "expiresAt": "2026-02-01T18:00:00Z"
}
```

Files

21. Upload File

```http
POST /api/v1/sessions/:sessionId/files
```

```
Authorization: Bearer {token}
Content-Type: multipart/form-data

Form Data:
- file: (binary)

Response: 201 Created
{
  "id": "uuid",
  "sessionId": "uuid",
  "fileName": "homework.pdf",
  "fileUrl": "https://s3.amazonaws.com/.../homework.pdf",
  "fileSize": 102400,
  "fileType": "application/pdf",
  "uploadedBy": "uuid",
  "createdAt": "2026-02-01T10:20:00Z"
}

Errors:
- 400: File too large (max 10MB)
- 400: Invalid file type
```

22. List Session Files

http

```
GET /api/v1/sessions/:sessionId/files
Authorization: Bearer {token}

Response: 200 OK
{
  "files": [
    {
      "id": "uuid",
      "fileName": "homework.pdf",
      "fileUrl": "https://s3.amazonaws.com/.../homework.pdf",
      "fileSize": 102400,
      "fileType": "application/pdf",
      "uploadedBy": {
        "id": "uuid",
        "name": "John Doe"
      },
      "createdAt": "2026-02-01T10:20:00Z"
    }
  ]
}
```

Admin

23. List All Sessions (Admin)

```http
GET /api/v1/admin/sessions?status=scheduled&date=2026-02-01
Authorization: Bearer {admin_token}

Response: 200 OK
{
  "sessions": [...all sessions with full details...],
  "total": 50
}
```

24. List All Users (Admin)

```http
GET /api/v1/admin/users?role=teacher&status=pending
Authorization: Bearer {admin_token}

Query Parameters:
- role: student/teacher/admin
- status: pending/approved/active/inactive

Response: 200 OK
{
  "users": [
    {
      "id": "uuid",
      "email": "teacher@example.com",
      "firstName": "Jane",
      "lastName": "Teacher",
      "role": "teacher",
      "isApproved": false,
      "isActive": true,
      "createdAt": "2026-01-25T10:00:00Z"
    }
  ],
  "total": 15
}
```

25. Approve Teacher

```http
```

```
PATCH /api/v1/admin/teachers/:id/approve
Authorization: Bearer {admin_token}

Request Body:
{
  "approved": true
}

Response: 200 OK
{
  "id": "uuid",
  "isApproved": true
}
```

26. Dashboard Stats (Admin)

http

```
GET /api/v1/admin/dashboard
Authorization: Bearer {admin_token}

Response: 200 OK
{
  "stats": {
    "totalUsers": 150,
    "activeStudents": 100,
    "approvedTeachers": 25,
    "pendingTeachers": 5,
    "sessionsToday": 12,
    "sessionsThisWeek": 78,
    "sessionsThisMonth": 320,
    "upcomingSessions": 45
  },
  "recentSessions": [...recent 10 sessions...],
  "recentUsers": [...recent 10 registrations...]
}
```

Notifications

27. List User Notifications

http

```
GET /api/v1/notifications?unread=true&page=1&limit=20
Authorization: Bearer {token}

Response: 200 OK
{
  "notifications": [
    {
      "id": "uuid",
      "type": "session_scheduled",
      "title": "New Session Scheduled",
      "message": "You have a math session on Feb 1 at 10:00 AM",
      "relatedSessionId": "uuid",
      "isRead": false,
      "createdAt": "2026-01-30T15:00:00Z"
    }
  ],
  "unreadCount": 5,
  "total": 25
}
```

28. Mark Notification as Read

```http
PATCH /api/v1/notifications/:id/read
Authorization: Bearer {token}

Response: 200 OK
{
  "id": "uuid",
  "isRead": true,
  "readAt": "2026-01-30T16:00:00Z"
}
```

29. Mark All as Read

```http
POST /api/v1/notifications/read-all
Authorization: Bearer {token}

Response: 200 OK
{
```

```
  "updated": 5
}
```

LiveKit

30. Generate LiveKit Token

```http
POST /api/v1/livekit/token
Authorization: Bearer {token}

Request Body:
{
  "sessionId": "uuid"
}

Response: 200 OK
{
  "token": "eyJhbGc...",
  "url": "wss://livekit.yourdomain.com",
  "roomName": "session-abc123"
}
```

🏗️ PROJECT STRUCTURE

Backend (Go)

```
backend/
├── cmd/
│   └── server/
│       └── main.go              # Application entry point
│
├── internal/
│   ├── config/
│   │   └── config.go            # Configuration management (Viper)
│   │
│   ├── models/
│   │   ├── user.go              # User model + DTOs
│   │   ├── teacher.go           # Teacher model + DTOs
│   │   ├── session.go           # Session model + DTOs
│   │   ├── message.go           # Message model + DTOs
```

```
│   │   ├── whiteboard.go          # Whiteboard model + DTOs
│   │   ├── file.go                # File model + DTOs
│   │   └── notification.go        # Notification model + DTOs
│   │
│   ├── repository/
│   │   ├── user_repository.go       # User database operations
│   │   ├── teacher_repository.go    # Teacher database operations
│   │   ├── session_repository.go    # Session database operations
│   │   ├── message_repository.go    # Message database operations
│   │   ├── whiteboard_repository.go # Whiteboard database operations
│   │   ├── file_repository.go       # File database operations
│   │   └── notification_repository.go # Notification database operations
│   │
│   ├── service/

│   │   ├── auth_service.go          # Authentication logic
│   │   ├── user_service.go          # User business logic
│   │   ├── teacher_service.go       # Teacher business logic
│   │   ├── session_service.go       # Session business logic
│   │   ├── message_service.go       # Message business logic
│   │   ├── whiteboard_service.go    # Whiteboard business logic
│   │   ├── file_service.go          # File upload/management
│   │   ├── notification_service.go  # Notification creation/sending
│   │   ├── livekit_service.go       # LiveKit token generation
│   │   └── email_service.go         # Email sending (AWS SES)
│   │
│   ├── handler/
│   │   ├── auth_handler.go          # Auth HTTP handlers
│   │   ├── user_handler.go          # User HTTP handlers
│   │   ├── teacher_handler.go       # Teacher HTTP handlers
│   │   ├── session_handler.go       # Session HTTP handlers
│   │   ├── message_handler.go       # Message HTTP handlers
│   │   ├── whiteboard_handler.go    # Whiteboard HTTP handlers
│   │   ├── file_handler.go          # File HTTP handlers
│   │   ├── notification_handler.go  # Notification HTTP handlers
│   │   ├── admin_handler.go         # Admin HTTP handlers
│   │   └── livekit_handler.go       # LiveKit HTTP handlers
│   │
│   ├── middleware/
│   │   ├── auth.go                # JWT authentication middleware
│   │   ├── admin.go               # Admin authorization middleware
│   │   ├── cors.go                # CORS middleware
│   │   ├── logger.go              # Request logging middleware
│   │   └── ratelimit.go           # Rate limiting middleware
│   │
│   ├── database/
│   │   ├── postgres.go            # PostgreSQL connection
│   │   └── redis.go               # Redis connection
```

```
│   │   redis.go              # Redis connection
│   │
│   └── websocket/
│       └── hub.go             # WebSocket hub for real-time updates
│
├── pkg/
│   ├── utils/
│   │   ├── jwt.go             # JWT utilities
│   │   ├── hash.go            # Password hashing
│   │   ├── validator.go       # Custom validators
│   │   └── response.go        # Standard response formatters
│   │
│   ├── errors/
│   │   └── errors.go          # Custom error types
│   │

│   └── constants/
│       └── constants.go       # Application constants
│
├── migrations/
│   ├── 001_create_users.up.sql
│   ├── 001_create_users.down.sql
│   ├── 002_create_teachers.up.sql
│   ├── 002_create_teachers.down.sql
│   ├── 003_create_sessions.up.sql
│   ├── 003_create_sessions.down.sql
│   ├── 004_create_session_messages.up.sql
│   ├── 004_create_session_messages.down.sql
│   ├── 005_create_whiteboard_states.up.sql
│   ├── 005_create_whiteboard_states.down.sql
│   ├── 006_create_session_files.up.sql
│   ├── 006_create_session_files.down.sql
│   ├── 007_create_notifications.up.sql
│   └── 007_create_notifications.down.sql
│
├── .env.example
├── .env
├── go.mod
├── go.sum
├── Dockerfile
├── Dockerfile.dev
├── Makefile
└── README.md
```

Frontend (Next.js)

```
frontend/
├── src/
```

```
│   ├── app/
│   │   ├── (auth)/
│   │   │   ├── login/
│   │   │   │   └── page.tsx          # Login page
│   │   │   └── register/
│   │   │       └── page.tsx          # Registration page
│   │   │
│   │   ├── (dashboard)/
│   │   │   ├── dashboard/
│   │   │   │   └── page.tsx          # User dashboard
│   │   │   │
│   │   │   ├── teachers/
│   │   │   │   ├── page.tsx          # Teachers list
│   │   │   │   └── [id]/
│   │   │   │       └── page.tsx      # Teacher detail
│   │   │   │
│   │   │   ├── sessions/
│   │   │   │   ├── page.tsx          # Sessions list
│   │   │   │   └── [id]/
│   │   │   │       └── page.tsx      # Session detail
│   │   │   │
│   │   │   ├── notifications/
│   │   │   │   └── page.tsx          # Notifications page
│   │   │   │
│   │   │   └── layout.tsx            # Dashboard layout
│   │   │
│   │   ├── (admin)/
│   │   │   ├── admin/
│   │   │   │   ├── dashboard/
│   │   │   │   │   └── page.tsx      # Admin dashboard
│   │   │   │   │
│   │   │   │   ├── sessions/
│   │   │   │   │   ├── page.tsx      # Admin sessions list
│   │   │   │   │   ├── new/
│   │   │   │   │   │   └── page.tsx   # Create session
│   │   │   │   │   └── calendar/
│   │   │   │   │       └── page.tsx   # Calendar view
│   │   │   │   │
│   │   │   │   ├── users/
│   │   │   │   │   └── page.tsx      # Users management
│   │   │   │   │
│   │   │   │   ├── teachers/
│   │   │   │   │   └── page.tsx      # Teacher approvals
│   │   │   │   │
│   │   │   │   └── analytics/
│   │   │   │       └── page.tsx      # Analytics dashboard
│   │   │   │
```

```
│   │   │   └── layout.tsx          # Admin layout
│   │   │
│   │   ├── session/
│   │   │   └── [id]/
│   │   │       └── page.tsx        # Video session room
│   │   │
│   │   ├── layout.tsx              # Root layout
│   │   ├── page.tsx                # Landing page
│   │   └── globals.css             # Global styles
│   │
│   ├── components/
│   │   ├── ui/                     # Shadcn/UI components
│   │   │   ├── button.tsx
│   │   │   ├── input.tsx
│   │   │   ├── dialog.tsx

│   │   │   ├── card.tsx
│   │   │   ├── badge.tsx
│   │   │   ├── avatar.tsx
│   │   │   ├── dropdown-menu.tsx
│   │   │   ├── calendar.tsx
│   │   │   ├── table.tsx
│   │   │   ├── tabs.tsx
│   │   │   ├── toast.tsx
│   │   │   └── ...
│   │   │
│   │   ├── auth/
│   │   │   ├── LoginForm.tsx       # Login form
│   │   │   ├── RegisterForm.tsx    # Registration form
│   │   │   └── ProtectedRoute.tsx  # Auth guard component
│   │   │
│   │   ├── session/
│   │   │   ├── VideoRoom.tsx       # LiveKit video room
│   │   │   ├── PreJoinScreen.tsx   # Pre-join device check
│   │   │   ├── SessionControls.tsx # Video controls
│   │   │   ├── Whiteboard.tsx      # Excalidraw whiteboard
│   │   │   ├── Chat.tsx            # Chat component
│   │   │   ├── ChatMessage.tsx     # Single message component
│   │   │   ├── FileShare.tsx       # File sharing component
│   │   │   └── SessionInfo.tsx     # Session info sidebar
│   │   │
│   │   ├── teacher/
│   │   │   ├── TeacherCard.tsx     # Teacher card component
│   │   │   ├── TeacherList.tsx     # Teachers list
│   │   │   └── TeacherFilter.tsx   # Filter component
│   │   │
│   │   ├── admin/
│   │   │   ├── SessionForm.tsx     # Create/edit session form
```

```
│   │   │   ├── SessionCalendar.tsx    # Calendar view component
│   │   │   ├── UserTable.tsx         # Users table
│   │   │   ├── StatsCard.tsx         # Dashboard stats card
│   │   │   └── AnalyticsChart.tsx    # Charts component
│   │   │
│   │   ├── notification/
│   │   │   ├── NotificationBell.tsx   # Notification bell icon
│   │   │   ├── NotificationList.tsx   # Notifications list
│   │   │   └── NotificationItem.tsx   # Single notification
│   │   │
│   │   └── layout/
│   │       ├── Navbar.tsx            # Top navigation
│   │       ├── Sidebar.tsx           # Sidebar navigation
│   │       ├── Footer.tsx            # Footer
│   │       └── DashboardShell.tsx    # Dashboard container

│   │
│   ├── lib/
│   │   ├── api.ts                  # API client (fetch wrapper)
│   │   ├── auth.ts                 # Auth utilities
│   │   ├── utils.ts                # General utilities
│   │   ├── validations.ts          # Zod validation schemas
│   │   ├── websocket.ts            # WebSocket client
│   │   └── constants.ts            # Frontend constants
│   │
│   ├── hooks/
│   │   ├── useAuth.ts              # Auth hook
│   │   ├── useSessions.ts          # Sessions query hooks
│   │   ├── useTeachers.ts          # Teachers query hooks
│   │   ├── useMessages.ts          # Messages query hooks
│   │   ├── useNotifications.ts     # Notifications hook
│   │   ├── useWebSocket.ts         # WebSocket hook
│   │   └── useWhiteboard.ts        # Whiteboard sync hook
│   │
│   ├── store/
│   │   ├── authStore.ts            # Auth Zustand store
│   │   ├── sessionStore.ts         # Session state store
│   │   └── notificationStore.ts    # Notification state store
│   │
│   └── types/
│       ├── api.ts                  # API response types
│       ├── session.ts              # Session types
│       ├── user.ts                 # User types
│       ├── message.ts              # Message types
│       └── whiteboard.ts           # Whiteboard types
│
├── public/
│   ├── images/
```

```
│   └── icons/
│
├── .env.local
├── .env.local.example
├── next.config.js
├── tailwind.config.ts
├── tsconfig.json
├── package.json
├── postcss.config.js
└── README.md
```

📦 DEVELOPMENT PHASES

PHASE 1: Foundation & Authentication (Weeks 1-3)

Week 1: Project Setup & Database

Backend Tasks:

1. Initialize Go project
2. Set up project structure
3. Configure Viper for environment variables
4. Set up PostgreSQL connection with GORM
5. Set up Redis connection
6. Create all database migrations
7. Implement database connection pooling
8. Set up logging

Frontend Tasks:

1. Initialize Next.js project with TypeScript
2. Set up Tailwind CSS
3. Install and configure Shadcn/UI
4. Set up project structure
5. Configure environment variables
6. Set up API client with fetch wrapper
7. Install React Query and Zustand

Docker Tasks:

1. Create docker-compose.yml for development
2. Set up PostgreSQL container

2. Set up PostgreSQL container
3. Set up Redis container
4. Set up LiveKit container
5. Create backend Dockerfile.dev
6. Create frontend Dockerfile.dev

Deliverables:

- ✅ Development environment working
- ✅ Database migrations run successfully
- ✅ All services accessible via Docker

Week 2-3: Authentication System

Backend Tasks:

1. Implement User model and DTOs
2. Implement Teacher model and DTOs
3. Create user repository with CRUD operations
4. Create teacher repository
5. Implement JWT token generation and validation
6. Implement password hashing with bcrypt
7. Create auth service (register, login)
8. Create auth handler with endpoints
9. Implement JWT middleware
10. Implement admin middleware
11. Add input validation

Frontend Tasks:

1. Create auth store with Zustand
2. Create API client authentication methods
3. Build LoginForm component
4. Build RegisterForm component
5. Create login page
6. Create registration page
7. Implement protected route component
8. Create auth context/provider
9. Build basic navbar with auth state
10. Add form validation with Zod

10. Add form validation with Zod

Testing:

- ✅ User can register as student
- ✅ User can register as teacher
- ✅ User can login
- ✅ JWT token is generated and stored
- ✅ Protected routes work correctly
- ✅ Teacher requires approval before access

Deliverables:

- ✅ Complete authentication system
- ✅ User registration and login working
- ✅ JWT middleware protecting routes
- ✅ Teacher approval workflow in place

PHASE 2: Session Management & Admin Panel (Weeks 4-6)

Week 4: Session Data Layer

Backend Tasks:

1. Implement Session model and DTOs
2. Create session repository
3. Implement session service with:
    - Create session (admin only)
    - Get user sessions
    - Get session by ID
    - Update session
    - Cancel session
    - Check availability
    - Prevent double-booking
4. Create session handler
5. Add session validation logic
6. Implement session status transitions

Frontend Tasks:

1. Create session types
2. Build useSessions hook with React Query
3. Create SessionCard component
4. Create SessionList component
5. Build sessions page
6. Build session detail page
7. Add session status badges

Testing:

- ✅ Admin can create sessions
- ✅ System prevents double-booking
- ✅ Users can view their sessions
- ✅ Session status updates correctly

Week 5: Teacher Management

Backend Tasks:

1. Implement teacher listing with filters
2. Implement teacher approval endpoint
3. Create admin/teachers approval handler
4. Add teacher profile endpoints

Frontend Tasks:

1. Create useTeachers hook
2. Build TeacherCard component
3. Build TeacherList component
4. Build TeacherFilter component
5. Create teachers page
6. Create teacher detail page
7. Build teacher approval UI (admin)

Testing:

- ✅ Students can browse approved teachers
- ✅ Admin can approve/reject teachers
- ✅ Teacher profiles display correctly

Week 6: Admin Panel

Backend Tasks:

1. Create admin dashboard stats endpoint
2. Implement admin users list endpoint
3. Implement admin sessions list endpoint
4. Add analytics endpoints

Frontend Tasks:

1. Create admin layout
2. Build admin dashboard page with stats
3. Build SessionForm component for creating sessions
4. Build admin sessions list page
5. Build admin users management page
6. Build admin teacher approvals page
7. Create StatsCard components
8. Implement calendar view for sessions
9. Add date/time picker for scheduling

Testing:

- ✅ Admin can view dashboard stats
- ✅ Admin can create and assign sessions
- ✅ Admin can view all users
- ✅ Admin can approve teachers
- ✅ Calendar view shows all sessions

Deliverables:

- ✅ Complete session management system
- ✅ Teacher listing and approval
- ✅ Full admin panel with calendar view
- ✅ Session creation and assignment working

PHASE 3: Video Integration (Weeks 7-9)

Week 7: LiveKit Setup & Basic Video

Backend Tasks:

Backend Tasks:

1. Install LiveKit server SDK
2. Implement livekit service with token generation
3. Create livekit handler
4. Implement session join endpoint
5. Add session join validation (timing, permissions)
6. Implement session status updates
7. Set up LiveKit server in Docker

Frontend Tasks:

1. Install @livekit/components-react
2. Create PreJoinScreen component
3. Implement device selection (camera/mic)
4. Build basic VideoRoom component
5. Integrate LiveKit Room component
6. Create SessionControls component
7. Implement mute/unmute functionality
8. Implement camera on/off
9. Create end call functionality
10. Add session page routing
11. Handle disconnection

Testing:

- ✅ LiveKit server running
- ✅ Tokens generated correctly
- ✅ Users can join video session
- ✅ Camera and mic work
- ✅ Controls functional
- ✅ Session ends properly

Week 8: Enhanced Video Features

Backend Tasks:

1. Implement WebSocket hub for real-time updates
2. Add presence tracking
3. Implement typing indicators backend
4. Add read receipts logic

4. Add read receipts logic

Frontend Tasks:

1. Implement screen sharing
2. Add participant list display
3. Implement connection quality indicator
4. Add network stats display
5. Create SessionInfo sidebar
6. Implement WebSocket connection
7. Add real-time presence indicators
8. Handle reconnection logic
9. Add loading states
10. Improve error handling

Testing:

- ✅ Screen sharing works
- ✅ Participant list updates in real-time
- ✅ Connection quality displayed
- ✅ Reconnection works smoothly

Week 9: Video Polish & Testing

Frontend Tasks:

1. Add full-screen mode
2. Implement picture-in-picture
3. Add keyboard shortcuts
4. Improve video layout responsiveness
5. Add session timer
6. Implement "session starting soon" notification
7. Add pre-call test page
8. Optimize video quality settings
9. Add bandwidth detection

Testing:

- ✅ Video quality good on various networks
- ✅ All controls work reliably
- ✅ UI responsive and intuitive

- ✅ No major bugs

Deliverables:

- ✅ Full video conferencing working
- ✅ LiveKit integrated and stable
- ✅ Screen sharing functional
- ✅ Connection quality monitoring
- ✅ Real-time presence indicators

PHASE 4: Chat System (Weeks 10-11)

Week 10: Chat Implementation

Backend Tasks:

1. Implement Message model and repository
2. Create message service with:
    - Send message
    - Get session messages
    - Mark as read
    - Search messages
3. Create message handler
4. Implement WebSocket message broadcasting
5. Add typing indicators
6. Add read receipts

Frontend Tasks:

1. Build Chat component
2. Create ChatMessage component
3. Implement message input
4. Add emoji picker
5. Implement typing indicators (frontend)
6. Add read receipts display
7. Create message timestamp formatting
8. Add auto-scroll to new messages
9. Implement message search

10. Add unread message count

Testing:

- ✅ Messages send and receive instantly
- ✅ Typing indicators work
- ✅ Read receipts update correctly
- ✅ Message history loads properly
- ✅ Search works

Week 11: File Sharing in Chat

Backend Tasks:

1. Implement file upload to S3
2. Create file model and repository
3. Implement file service
4. Create file handler
5. Add file validation (type, size)
6. Generate signed URLs for downloads
7. Implement file deletion

Frontend Tasks:

1. Build FileShare component
2. Implement file picker
3. Add file upload progress bar
4. Create file preview (images, PDFs)
5. Implement file download
6. Add file type icons
7. Display file size formatting
8. Add drag-and-drop file upload

Testing:

- ✅ Files upload to S3
- ✅ File links work
- ✅ File types validated
- ✅ Progress bar works
- ✅ Downloads work

Deliverables:

- ✅ Full chat system working
- ✅ Typing indicators and read receipts
- ✅ Message search
- ✅ File sharing functional
- ✅ Chat history persisted

PHASE 5: Whiteboard Integration (Weeks 12-13)

Week 12: Excalidraw Integration

Backend Tasks:

1. Implement whiteboard_states model and repository
2. Create whiteboard service
3. Implement save whiteboard state endpoint
4. Implement load whiteboard state endpoint
5. Add versioning for whiteboard states
6. Implement whiteboard export to S3

Frontend Tasks:

1. Install @excalidraw/excalidraw
2. Create Whiteboard component
3. Integrate Excalidraw with custom UI
4. Implement whiteboard state sync via LiveKit DataChannel
5. Add save whiteboard button
6. Add load whiteboard button
7. Implement real-time collaboration
8. Handle whiteboard initialization

Testing:

- ✅ Whiteboard loads
- ✅ Both users can draw simultaneously
- ✅ Changes sync in real-time
- ✅ Whiteboard state saves
- ✅ Whiteboard state loads on rejoin

- ✅ Whiteboard state loads on rejoin

Week 13: Whiteboard Features & Export

Frontend Tasks:

1. Add whiteboard toolbar customization
2. Implement color picker
3. Add shape tools integration
4. Implement text tool
5. Add image upload to whiteboard
6. Implement undo/redo
7. Add clear canvas option
8. Create export functionality (PNG/SVG)
9. Add whiteboard zoom controls
10. Optimize performance

Backend Tasks:

1. Implement export endpoint
2. Generate image from whiteboard data
3. Upload export to S3
4. Return signed URL

Testing:

- ✅ All drawing tools work
- ✅ Image upload works
- ✅ Export generates correct images
- ✅ Performance is good
- ✅ No sync issues

Deliverables:

- ✅ Full whiteboard functionality
- ✅ Real-time collaboration working
- ✅ Whiteboard state persistence
- ✅ Export feature working
- ✅ Image upload to whiteboard

PHASE 6: Notifications & Polish (Weeks 14-16)

Week 14: Notification System

Backend Tasks:

1. Implement Notification model and repository
2. Create notification service
3. Implement notification creation on events:
    - Session scheduled
    - Session starting soon (15 min before)
    - Session cancelled
    - Session rescheduled
    - Teacher approved
4. Create notification handler
5. Implement WebSocket notification push
6. Set up email service with AWS SES
7. Create email templates
8. Implement email sending for notifications

Frontend Tasks:

1. Create useNotifications hook
2. Build NotificationBell component
3. Build NotificationList component
4. Build NotificationItem component
5. Implement real-time notification updates
6. Add unread count badge
7. Add mark as read functionality
8. Add mark all as read
9. Create notifications page
10. Add notification sound (optional)

Testing:

- ✅ Notifications created on events
- ✅ Real-time updates work
- ✅ Emails sent correctly
- ✅ Mark as read works
- ✅ Unread count accurate

Week 15: Admin Analytics & Calendar

Backend Tasks:

1. Implement analytics calculations
2. Add more dashboard stats
3. Optimize dashboard queries

Frontend Tasks:

1. Build SessionCalendar component with react-big-calendar
2. Implement calendar drag-and-drop (optional)
3. Create AnalyticsChart component
4. Add charts with recharts library
5. Implement date range filtering
6. Add export reports functionality (CSV)
7. Build session history timeline
8. Add filtering and sorting to tables

Testing:

- ✅ Calendar displays all sessions
- ✅ Charts render correctly
- ✅ Analytics data accurate
- ✅ Filtering works

Week 16: Final Polish, Testing & Deployment

Backend Tasks:

1. Add comprehensive error handling
2. Implement request validation on all endpoints
3. Add rate limiting
4. Optimize database queries
5. Add database indexes

5. Add database indexes
6. Implement connection pooling optimization
7. Add health check endpoint
8. Write API documentation

Frontend Tasks:

1. Add loading states everywhere
2. Improve error messages
3. Add empty states
4. Implement skeleton loaders
5. Optimize images
6. Add SEO meta tags
7. Implement responsive design fixes
8. Add accessibility improvements
9. Test on different browsers
10. Fix all bugs

DevOps Tasks:

1. Create production Docker files
2. Set up AWS infrastructure
3. Configure RDS PostgreSQL
4. Configure ElastiCache Redis
5. Set up S3 bucket
6. Configure CloudFront CDN
7. Set up AWS SES for email
8. Deploy LiveKit server
9. Set up CI/CD pipeline
10. Configure monitoring and alerts
11. Set up backup strategy
12. Create deployment documentation

Testing Checklist:

- ✅ All authentication flows work
- ✅ All user roles work correctly
- ✅ Session creation and assignment work
- ✅ Video calls stable and clear

- ✅ Whiteboard syncs properly
- ✅ Chat works reliably
- ✅ File sharing works
- ✅ Notifications delivered correctly
- ✅ Admin panel fully functional
- ✅ Calendar view works
- ✅ Analytics accurate
- ✅ No console errors
- ✅ Mobile responsive
- ✅ Cross-browser compatible
- ✅ Performance acceptable
- ✅ Security tested

Deliverables:

- ✅ Production-ready application
- ✅ All features complete and tested
- ✅ Deployed to AWS
- ✅ Documentation complete
- ✅ Monitoring in place

🚀 GETTING STARTED

Initial Setup Commands

Backend Setup

```bash
# Create project structure
mkdir -p backend/{cmd/server,internal/{config,models,repository,service,handler,middleware,database,websocket},pkg/{

# Initialize Go module
cd backend
go mod init tutoring-platform

# Install dependencies
go get github.com/gin-gonic/gin@v1.10.0
go get github.com/jackc/pgx/v5
go get gorm.io/gorm
go get gorm.io/driver/postgres
```

```
go get gorm.io/driver/postgres
go get github.com/redis/go-redis/v9
go get github.com/golang-jwt/jwt/v5
go get github.com/go-playground/validator/v10
go get github.com/spf13/viper
go get github.com/livekit/server-sdk-go
go get github.com/golang-migrate/migrate/v4
go get golang.org/x/crypto/bcrypt
go get github.com/google/uuid
go get github.com/aws/aws-sdk-go-v2/config
go get github.com/aws/aws-sdk-go-v2/service/s3
go get github.com/aws/aws-sdk-go-v2/service/ses
```

Frontend Setup

```bash
# Create Next.js app
npx create-next-app@14.2.0 frontend --typescript --tailwind --app --src-dir --import-alias "@/*"

cd frontend

# Install dependencies
npm install @tanstack/react-query@5.0.0
npm install zustand@4.5.0
npm install react-hook-form@7.50.0
npm install zod@3.22.0
npm install @hookform/resolvers
npm install @livekit/components-react@2.0.0
npm install livekit-client
npm install @excalidraw/excalidraw@0.17.0
npm install date-fns@3.0.0
npm install lucide-react@0.300.0
npm install react-big-calendar
npm install recharts
npm install @aws-sdk/client-s3
npm install @aws-sdk/s3-request-presigner

# Install Shadcn/UI
npx shadcn-ui@latest init
npx shadcn-ui@latest add button
npx shadcn-ui@latest add input
npx shadcn-ui@latest add dialog
```

```
npx shadcn-ui@latest add dialog
npx shadcn-ui@latest add card
npx shadcn-ui@latest add badge
npx shadcn-ui@latest add avatar
npx shadcn-ui@latest add dropdown-menu
npx shadcn-ui@latest add calendar
npx shadcn-ui@latest add table
npx shadcn-ui@latest add tabs
npx shadcn-ui@latest add toast
```

Docker Setup

Create `docker-compose.yml` in project root:

```yaml
version: '3.8'

services:
  postgres:
    image: postgres:15-alpine
    container_name: tutoring-postgres
    environment:
      POSTGRES_DB: tutoring_db
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 10s
      timeout: 5s
      retries: 5

  redis:
    image: redis:7-alpine
    container_name: tutoring-redis
    ports:
      - "6379:6379"
    volumes:
      - redis_data:/data
    command: redis-server --appendonly yes

  livekit:
    image: livekit/livekit-server:latest
    container_name: tutoring-livekit
    command: --config /etc/livekit.yaml
```

```yaml
    command: --config /etc/livekit.yaml
    ports:
      - "7880:7880"
      - "7881:7881"
      - "7882:7882/udp"
      - "50000-50100:50000-50100/udp"
    volumes:
      - ./livekit.yaml:/etc/livekit.yaml
    environment:
      - LIVEKIT_KEYS=devkey: secret123
    depends_on:
      - redis

  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile.dev
    container_name: tutoring-backend
    environment:
      - DATABASE_URL=postgres://postgres:postgres@postgres:5432/tutoring_db?sslmode=disable
      - REDIS_URL=redis:6379
      - JWT_SECRET=your-super-secret-jwt-key-change-in-production
      - LIVEKIT_URL=ws://livekit:7880
      - LIVEKIT_API_KEY=devkey
      - LIVEKIT_API_SECRET=secret123
      - AWS_REGION=ap-south-1
      - AWS_ACCESS_KEY_ID=your_access_key

      - AWS_SECRET_ACCESS_KEY=your_secret_key
      - AWS_S3_BUCKET=your-bucket-name
      - PORT=8080
    ports:
      - "8080:8080"
    volumes:
      - ./backend:/app
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_started
      livekit:
        condition: service_started
    command: go run cmd/server/main.go

  frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile.dev
    container_name: tutoring-frontend
```

```yaml
    container_name: tutoring-frontend
    environment:
      - NEXT_PUBLIC_API_URL=http://localhost:8080
      - NEXT_PUBLIC_LIVEKIT_URL=ws://localhost:7880
    ports:
      - "3000:3000"
    volumes:
      - ./frontend:/app
      - /app/node_modules
      - /app/.next
    depends_on:
      - backend
    command: npm run dev

volumes:
  postgres_data:
  redis_data:
```

Create livekit.yaml:

```yaml
port: 7880

rtc:
  port_range_start: 50000
  port_range_end: 50100
  use_external_ip: true

redis:
  address: redis:6379

keys:
  devkey: secret123

room:
  auto_create: true
  empty_timeout: 300
  max_participants: 2

logging:
  level: info
```

⚠️ CRITICAL RULES FOR AI AGENT

DO NOT Add These Features

1. ❌ NO payment processing (Razorpay, Stripe, PayPal, etc.)
2. ❌ NO automated booking by students
3. ❌ NO mobile apps
4. ❌ NO session recording
5. ❌ NO SMS notifications
6. ❌ NO social login (Google OAuth, Facebook login, etc.)
7. ❌ NO quiz/assessment system
8. ❌ NO homework submission
9. ❌ NO parent portal
10. ❌ NO certificates
11. ❌ NO AI features (recommendations, auto-grading, etc.)
12. ❌ NO video filters or virtual backgrounds
13. ❌ NO multiple user roles beyond student/teacher/admin
14. ❌ NO public API access
15. ❌ NO third-party integrations (Zoom, Google Calendar, etc.)

Must Follow Exactly

1. ✅ Use exact technology versions specified
2. ✅ Use exact database schema provided
3. ✅ Use exact API endpoints listed
4. ✅ Follow exact project structure
5. ✅ Implement features in exact phase order
6. ✅ Use exact naming conventions
7. ✅ Implement exact error handling
8. ✅ Use exact authentication method (JWT)
9. ✅ Use exact WebSocket for real-time (not Socket.io)
10. ✅ Use exact libraries specified (no alternatives)

Security Requirements